실시간 객체 탐지 파이프라인

로봇 카메라 · YOLO · 칼만 필터 · 최종 실습

cchyun@gmail.com

Part 1. 전체 파이프라인 구조

최종 실습 목표 — 지능형 객체 탐지 및 추적 시스템 구축

- **로봇이 사물을 보고 판단하기 위한 4단계 통합 파이프라인을 구현**

  - **대상:** 파이썬 입문자 대상 2시간 완성형 실습
  - **목표:** 카메라 입력 → 왜곡 보정 → 탐지 → 추적 → 시각화까지 전체 흐름 직접 구현

  - **최종 결과물**
    - 웹캠 영상에서 사람, 컵, 휴대폰 등을 실시간으로 탐지하고
    - 칼만 필터로 움직임을 안정적으로 추적하는 로봇 비전 시스템

4단계 파이프라인 구성

- **4단계 통합 처리 흐름**

  - **왜곡 보정 (Undistortion)**
    - 카메라 렌즈의 특성으로 발생하는 휘어짐 현상을 캘리브레이션 데이터를 이용해 평평하게 펌

  - **객체 탐지 (Object Detection)**
    - YOLOv8 딥러닝 모델을 사용하여 실시간 영상에서 사물의 위치와 종류를 찾아냄

```
[웹캠 입력] → [왜곡 보정] → [YOLO 탐지] → [칼만 필터 추적] → [결과 시각화]
```

4단계 파이프라인 구성 (계속)

- **이후 처리 흐름**

  - **객체 추적 (Kalman Filter Tracking)**
    - 탐지된 위치의 노이즈를 제거하고
    - 객체가 잠시 가려지더라도 다음 위치를 예측하여 안정적으로 추적

  - **결과 시각화 (Visualization)**
    - 보정된 영상 위에 탐지된 바운딩 박스(Bounding Box)를 그려 화면에 표시
    - 칼만 필터 예측 위치를 별도 마커로 표시

Part 2. 단계별 구현 순서

점진적 구현 방식 — 기능 하나씩 추가하며 완성

- **입문자 눈높이에 맞춰 기능을 하나씩 추가하며 완성해 나갑니다**
  - **Step 1:** 라이브러리 설치 및 기본 웹캠 출력 확인
    - 화면에 영상이 끊김 없이 출력되는지 먼저 검증
  - **Step 2:** 카메라 캘리브레이션 상수를 적용한 왜곡 보정 기능 추가
    - `cv2.undistort()` 로 렌즈 왜곡 제거
  - **Step 3:** YOLOv8 모델을 통한 실시간 사물 인지 연동
    - 탐지된 객체의 바운딩 박스 좌표 추출

점진적 구현 방식 (계속)

- **후반부 단계에서 추적과 시각화를 완성**
  - **Step 4:** 칼만 필터(Kalman Filter) 클래스 정의 및 추적 로직 삽입
    - `SimpleKalman` 클래스를 만들어 노이즈 있는 탐지 좌표를 보정
  - **Step 5:** 최종 통합 및 시각화 꾸미기
    - 탐지 박스(파란색)와 칼만 예측 점(빨간색)을 영상에 표시
    - 레이블 텍스트 추가

Part 3. 완성 코드 구현

SimpleKalman 클래스 — 칼만 필터 구현 (1/2)

```python
# [필수] 칼만 필터 기초 클래스 (단순화 버전)
class SimpleKalman:
    def __init__(self):
        # 칼만 필터 객체 생성
        # 상태 변수 4개(x, y, dx, dy), 측정 변수 2개(x, y)
        self.kf = cv2.KalmanFilter(4, 2)

        # 측정 행렬: 측정값(x, y)과 상태 벡터를 연결
        self.kf.measurementMatrix = np.array(
            [[1, 0, 0, 0],
             [0, 1, 0, 0]], np.float32)

        # 전이 행렬: 등속 운동 모델 (x, y, dx, dy)
        self.kf.transitionMatrix = np.array(
            [[1, 0, 1, 0],
             [0, 1, 0, 1],
             [0, 0, 1, 0],
             [0, 0, 0, 1]], np.float32)
```

SimpleKalman 클래스 — 칼만 필터 구현 (2/2)

```
        # 프로세스 노이즈 공분산 행렬
        # 값이 클수록 모델 예측보다 측정값을 더 신뢰
        self.kf.processNoiseCov = np.eye(4, dtype=np.float32) * 0.03

    def predict(self, coord_x, coord_y):
        measured = np.array(
            [[np.float32(coord_x)],
             [np.float32(coord_y)]])

        self.kf.predict()                # 예측 단계
        corrected = self.kf.correct(measured)  # 업데이트 단계

        # 보정된 x, y 좌표 반환
        return int(corrected[0]), int(corrected[1])
```

메인 코드 — 환경 설정 및 모델 로드 (1/3)

```python
# 1. YOLO 모델 로드 (초경량 Nano 모델)
model = YOLO('yolov8n.pt')

# 캘리브레이션 파라미터 로드
YAML_PATH = "camera_info.yaml"
if not os.path.exists(YAML_PATH):
    raise FileNotFoundError(f"{YAML_PATH} 파일이 없습니다.")

with open(YAML_PATH, 'r') as f:
    _calib = yaml.safe_load(f)

# 카메라 매트릭스: fx, fy(초점 거리), cx, cy(주점)
camera_matrix = np.array(_calib['camera_matrix'], dtype=np.float32)
# 왜곡 계수: [k1, k2, p1, p2, k3]
dist_coeffs   = np.array(_calib['dist_coeff'],    dtype=np.float32)

# 웹캠 연결 (0번: 내장 웹캠)
cap = cv2.VideoCapture(0)

# 칼만 필터 추적기 생성
tracker = SimpleKalman()
```

메인 코드 — 왜곡 보정 및 YOLO 탐지 (2/3)

```python
while cap.isOpened():
    success, frame = cap.read()
    if not success:
        break

    # 2. 왜곡 보정 적용 (Undistort)
    # 카메라 고유 파라미터를 사용하여 렌즈 왜곡을 제거
    undistorted_frame = cv2.undistort(
        frame, camera_matrix, dist_coeffs)

    # 3. YOLOv8 객체 탐지
    results = model(undistorted_frame, stream=True, verbose=False)

    for r in results:
        boxes = r.boxes
        for box in boxes:
            # 바운딩 박스 좌표 추출 (x1, y1, x2, y2)
            x1, y1, x2, y2 = map(int, box.xyxy[0])

            # 객체 중심점 계산
            cx = (x1 + x2) // 2
            cy = (y1 + y2) // 2
```

메인 코드 — 칼만 필터 추적 및 시각화 (3/3)

```
            # 4. 칼만 필터 추적
            # 노이즈가 섞인 감지 위치를 보정하여 부드러운 추적 실현
            px, py = tracker.predict(cx, cy)

            # 5. 시각화
            cls = int(box.cls[0])
            label = model.names[cls]

            # 탐지 박스 (파란색)
            cv2.rectangle(undistorted_frame,
                          (x1, y1), (x2, y2), (255, 0, 0), 2)
            # 칼만 필터 예측 점 (빨간색)
            cv2.circle(undistorted_frame, (px, py), 5, (0, 0, 255), -1)
            cv2.putText(undistorted_frame, f"{label} (Tracked)",
                        (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX,
                        0.5, (255, 255, 255), 2)

    cv2.imshow("Robot Vision Pipeline", undistorted_frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Part 4. 심화 미션

미션 1 — 탐지된 객체까지의 거리 추정

- **핀홀 카메라 모델의 원리를 이용한 거리 계산**
  - **방법:** 사물의 실제 크기와 카메라의 초점 거리(f_x)를 알면 거리 추정 가능
  - **수식:**

$$Distance = \frac{\text{실제 높이} \times \text{조점 거리}}{\text{이미지상의 픽셀 높이}}$$

  - **코드 예시:**

```
REAL_HEIGHT  = 1.7            # 사람의 평균 키 (미터)
def estimate_distance(frame, x1, y1, x2, y2):
    focal_length = camera_matrix[1][1]    # fy 값
    pixel_height = y2 - y1                # 바운딩 박스 높이 (픽셀)
    if pixel_height <= 0:
        return None
    distance = (REAL_HEIGHT * focal_length) / pixel_height
    cv2.putText(frame, f"Dist: {distance:.1f}m",
                (x1, y2 + 20), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
    return distance
```

미션 2 — 특정 클래스 탐지 시 경고 출력

- **특정 사물이 탐지되면 화면에 경고 메시지를 표시**
  - **방법:** `label` 값을 확인하여 특정 클래스 탐지 시 텍스트 출력
  - **코드 예시:**

```
TARGET_LABEL = "cell phone"  # 경고를 발생시킬 클래스 이름
def check_warning(frame, label):
    if label == TARGET_LABEL:
        cv2.putText(frame, f"WARNING: {TARGET_LABEL} Detected!",
                    (50, 80), cv2.FONT_HERSHEY_SIMPLEX, 1.5, (0, 0, 255), 3)
```

  - **응용:** 특정 구역에서 특정 물체 탐지 → 알람, 로봇 정지 신호 등과 연동 가능

미션 3 — 탐지 결과를 JSON 파일로 저장

- **매 프레임 탐지된 객체의 이름과 좌표를 JSON 파일로 기록**
  - **방법:** 탐지 결과를 리스트에 담아 `json` 라이브러리로 파일에 기록
  - **코드 예시:**

```
detection_data = []

def save_detection(label, x1, y1, x2, y2, cx, cy):
    detection_data.append({
        "label": label,
        "bbox": [x1, y1, x2, y2],
        "center": [cx, cy]
    })

def save_detection_log(path="detection_log.json"):
    json.dump(detection_data, open(path, 'w'), ensure_ascii=False, indent=2)
    print(f"저장 완료: {path}  ({len(detection_data)}건)")
```

Part 5. P2 마무리 — 핵심 개념 정리

Day 4 핵심 개념 정리

- **[Day 4: 이미지 처리의 기초와 전처리]**
  - **이미지의 디지털 표현 (NumPy Array)**
    - 컴퓨터는 이미지를 **높이, 너비, 채널(BGR)** 을 가진 다차원 배열로 인식
  - **색상 공간 변환 (Color Spaces)**
    - BGR 이미지를 **그레이스케일(Grayscale)** 이나 조명 변화에 강한 **HSV** 공간으로 변환
  - **이미지 필터링과 블러링 (Gaussian Blur)**
    - 영상의 노이즈를 제거하여 에지 검출 등의 다음 단계 정확도를 높이는 전처리 핵심

Day 5 핵심 개념 정리

- [Day 5: 고급 탐지 기술과 상태 추정]
  - **에지 및 윤곽선 검출 (Canny & Contours)**
    - 픽셀 값의 급격한 변화를 감지하는 **Canny 알고리즘** 으로 사물의 테두리를 찾고 형태 분석
  - **실시간 객체 탐지 (YOLO)**
    - 딥러닝 기반의 **YOLO(You Only Look Once)** 로 실시간으로 사물의 분류와 위치를 한 번에 파악
  - **칼만 필터 (Kalman Filter)**
    - 센서 노이즈 속에서도 물리적 모델과 측정값을 결합하여
    - **예측(Predict)** 과 **업데이트(Update)** 과정을 통해 대상의 위치를 안정적으로 추적